

Dashboard Executivo Uniodonto

Diagnóstico profissional pré-entrega & plano de melhorias

5,1/10

NOTA GERAL DE PRONTIDÃO

Veredito: Base técnica muito boa e interface de nível profissional, mas **ainda NÃO está pronto para entrega**. Existem 4 bloqueadores críticos de segurança que precisam ser corrigidos antes de colocar dados reais da Uniodonto no ar.

Projeto: Uniodonto BI · Painel de Business Intelligence e Gestão de Vendas

Stack: React 19 · TypeScript · Vite · Tailwind · Firebase · Express

Data da avaliação: 04/07/2026

1. Sumário Executivo

O aplicativo é um painel executivo (BI) para acompanhamento de vendas, beneficiários, funil de marketing e indicadores financeiros da Uniodonto. A avaliação analisou o código-fonte completo, a arquitetura, as regras de segurança do Firebase, a interface e o preparo para produção.

Pontos fortes

- Interface polida, responsiva e com identidade visual da Uniodonto.
- TypeScript compila **sem nenhum erro**; código bem tipado.
- **Zero vulnerabilidades** nas dependências (npm audit).
- PWA funcional (instalável, service worker, ícones).
- Consentimento LGPD presente na tela de login.

Bloqueadores críticos

- Banco Firebase **totalmente aberto** (qualquer um lê/grava tudo).
- **Backdoor de acesso**: 3 cliques no logo entram como diretoria.
- Senha/API **fixas no código-fonte** (admin123 / chave "prod").
- Integração com Google Sheets **não implementada** (só dados fictícios).

Recomendação: não entregar para uso real da Uniodonto até que os itens da **Área 1 (Segurança)** e a **Área 2 (Integração de Dados)** estejam resolvidos. Os 4 bloqueadores críticos são resolvíveis em poucas horas de trabalho cada.

2. As áreas que tornam um app profissional

Estas são as 12 áreas usadas para medir se um aplicativo está pronto para produção. A nota de cada uma é real, baseada no que existe hoje no código.

#	Área	Nota	Nível	Situação
1	Segurança & Autenticação	2,5		CRÍTICO
2	Backend & Integração de Dados	4,0		Incompleto
3	Arquitetura & Qualidade de Código	6,5		Bom
4	UI/UX & Design Visual	8,5		Excelente
5	Responsividade & PWA	8,0		Excelente
6	Acessibilidade (a11y)	5,0		Regular
7	Testes Automatizados & CI	1,0		Ausente
8	Tratamento de Erros & Confiabilidade	5,5		Regular
9	LGPD & Privacidade	5,0		Atenção
10	DevOps, Build & Deploy	6,0		Bom
11	Documentação	7,0		Bom
12	Performance	7,0		Bom

Nota geral (média ponderada, com peso maior para segurança, dados e testes): **5,1 / 10**. A interface excelente (8,5) é puxada para baixo pelos bloqueadores de segurança (2,5) e ausência de testes (1,0).

3. Análise detalhada por área

Área 1 · Segurança & Autenticação — 2,5/10 **CRÍTICO**

Problemas encontrados:

- **Regras do Firestore e do Storage abertas:** `allow read, write: if true`. Qualquer pessoa na internet consegue ler e apagar todos os dados, inclusive os hashes de senha dos usuários.
- **Backdoor de login:** clicar 3× no logo autentica automaticamente como `diretoria@uniodonto.com / admin123` (visível no código-fonte).
- **Segredos fixos no código:** senha `admin123` e chave `uniodonto_prd_key_9582098410298410924` escritas diretamente nos arquivos.
- **Autenticação frágil:** senha é convertida em SHA-256 *no navegador*, sem "sal", e guardada em `localStorage` + Firestore aberto. Isso não é seguro. O ideal é usar o Firebase Authentication (e-mail/senha) de verdade.
- **Sessão sem verificação:** o acesso é liberado lendo um objeto do `localStorage`, sem validar token — pode ser forjado.

Impacto: exposição de dados de beneficiários e da operação da Uniodonto. É o bloqueador nº 1.

Área 2 · Backend & Integração de Dados — 4,0/10 **Incompleto**

- O README define a integração com **Google Sheets via Service Account** como o coração do projeto, mas as rotas do servidor (`server/routes.ts`) só devolvem **dados fictícios (mock)** ou erro `501 Not implemented`.
- Há **dois caminhos de dados concorrentes** e desconectados: a API Express (mock, não usada pela tela) e a leitura direta do Firestore (`dashboardData.ts`, essa sim usada). Isso confunde a arquitetura.
- Os números do painel (10.320 beneficiários, ROI, NPS etc.) são **valores de exemplo** — não vêm de fonte real ainda.

Área 3 · Arquitetura & Qualidade de Código — 6,5/10 **Bom**

- Bom: componentes organizados, hooks, tipagem forte, TypeScript compila limpo.
- A melhorar: `Configuracoes.tsx` tem **1.782 linhas** (arquivo grande demais, difícil manter). Não há ESLint configurado (o comando "lint" só roda o compilador). Restam `console.log` e código morto (rotas mock não usadas).

Área 4 · UI/UX & Design — 8,5/10 Excelente

- Interface limpa, hierarquia visual clara, identidade Uniodonto (magenta #CD176D) bem aplicada.
- Navegação inferior no mobile e barra lateral no desktop; cartões de KPI legíveis; gráficos com Recharts.
- Pequenos ajustes possíveis: estados de "carregando/vazio/erro" mais explícitos em algumas telas.

Área 5 · Responsividade & PWA — 8,0/10 Excelente

- Layout adapta bem a celular, tablet e desktop; classes responsivas presentes em todas as telas.
- PWA completo: manifest, service worker e ícones (192/512, apple-touch). Instalável.

Área 6 · Acessibilidade — 5,0/10 Regular

- Poucos atributos `aria-*` (só nas telas Config e Envio). Faltam rótulos em ícones-botão, textos alternativos em imagens e foco visível para navegação por teclado.

Área 7 · Testes Automatizados & CI — 1,0/10 Ausente

- **Nenhum teste** no projeto (nem unitário, nem de integração) e nenhuma automação de CI. Para um app entregue a cliente, isso é um risco: qualquer alteração futura pode quebrar algo sem ninguém perceber.

Área 8 · Tratamento de Erros & Confiabilidade — 5,5/10 Regular

- Bom: há fallback local quando o Firebase falha, então a tela não quebra.
- A melhorar: erros mostrados via `alert()`; vários `catch` silenciosos; sem *Error Boundary* React global.

Área 9 · LGPD & Privacidade — 5,0/10 Atenção

- Bom: existe consentimento LGPD e campo `lgpdAcceptedAt`.
- Risco: guardar hash de senha e dados pessoais em banco aberto **viola a LGPD**. Resolver a Área 1 corrige a maior parte disso.

Áreas 10–12 · Deploy (6,0) · Documentação (7,0) · Performance (7,0)

- **Deploy:** Firebase Hosting e scripts de build configurados; porém foram versionados arquivos que não deveriam ir para o repositório (`firebase-debug.log` , imagens `.tmp-*`).
- **Documentação:** README claro em português e modelo de dados documentado.
- **Performance:** build Vite com cache de assets; ok. Reduzir o arquivo gigante de Configurações ajuda.

4. Comandos de melhoria — Prompts prontos para IA

Cole cada bloco abaixo em um agente de código (Claude Code, Cursor etc.), um de cada vez, na ordem de prioridade. Comece pelos **P0 CRÍTICO**.

P0 CRÍTICO 1. Fechar as regras do Firebase

```
# Segurança - Firestore & Storage
Reescreva firestore.rules e storage.rules para NÃO usar "if true".
Exija request.auth != null em toda leitura/escrita. Na coleção "accounts"
e "users", permita que cada usuário só acesse o próprio documento
(match /users/{uid} { allow read, write: if request.auth.uid == uid }).
Restrinja "organizations/uniodonto/**" a usuários autenticados do domínio
@uniodonto.com. Depois rode "firebase deploy --only firestore:rules,storage".
```

P0 CRÍTICO 2. Remover o backdoor de login

```
# src/components/LoginPage.tsx
Remova completamente o atalho de "3 cliques no logo" que faz login automático
como diretoria@uniodonto.com / admin123 (função handleLogoClick e a variável
passwordValue). Nenhuma credencial pode ficar escrita no código. Garanta que o
único caminho de acesso seja o formulário de login normal.
```

P0 CRÍTICO 3. Tirar segredos do código-fonte

```
# src/pages/Configuracoes.tsx e variáveis de ambiente
Remova a chave fixa "uniodonto_prd_key_9582098410298410924" do estado inicial
de apiKey. Nunca versiona chaves de produção: mova qualquer segredo para
variáveis de ambiente (.env, já no .gitignore) e leia via import.meta.env.
Se a chave já foi commitada no git, gere uma nova e invalide a antiga.
```

P0 CRÍTICO 4. Autenticação real com Firebase Auth

```
# src/lib/firebase.ts + fluxo de login
Substitua o login por senha caseiro (SHA-256 no navegador + localStorage) pelo
Firebase Authentication de e-mail/senha (createUserWithEmailAndPassword /
signInWithEmailAndPassword). Pare de armazenar passwordHash no Firestore.
Faça o controle de sessão pelo onAuthStateChanged e valide o token do usuário,
em vez de ler o perfil direto do localStorage.
```

P1 ALTA 5. Implementar a integração real de dados

```
# server/routes.ts
Implemente a leitura real da Google Sheets API pela Service Account (usando as
variáveis GOOGLE_* do .env), substituindo as respostas mock e os 501. Defina
um único caminho de dados: escolha entre (a) Firestore como fonte, ou (b) a API
Express, e remova o caminho duplicado/morto para evitar confusão.
```

P1 ALTA 6. Testes automatizados + CI

Testes

Configure Vitest + React Testing Library. Crie testes para: cálculo de investimento (calculateInvestmentTotal), transformação de dados do dashboard, e o fluxo de login (aceitar válido / rejeitar inválido). Adicione um workflow de CI (GitHub Actions) que roda "npm run lint" e os testes a cada push.

P2 MÉDIA 7. Qualidade de código e limpeza

Manutenibilidade

Configure ESLint + Prettier com regras para React/TypeScript. Quebre Configuracoes.tsx (1782 linhas) em componentes menores por seção. Remova console.log e código morto. Adicione um Error Boundary React global e troque os alert() de erro por notificações (toast) na interface.

P2 MÉDIA 8. Acessibilidade e repositório limpo

a11y + higiene do repo

Adicione aria-label a todos os botões-ícone, alt em imagens e foco visível por teclado; garanta contraste adequado. Remova do versionamento firebase-debug.log e os arquivos .tmp-*.png e adicione-os ao .gitignore.

5. Checklist técnico de prontidão para entrega

Para um desenvolvedor humano acompanhar item a item. Marque conforme concluir.

Bloqueadores — resolver antes de entregar (P0)

- ☐ Reescrever `firestore.rules` exigindo autenticação e escopo por usuário/organização.
- ☐ Reescrever `storage.rules` exigindo autenticação (fim do `if true`).
- ☐ Fazer deploy das novas regras e testar acesso não autenticado (deve ser negado).
- ☐ Remover o backdoor de 3 cliques no logo (`LoginPage.tsx`).
- ☐ Remover senha `admin123` e a chave `uniodonto_prd_key...` do código.
- ☐ Rotacionar/invalidar qualquer segredo que já tenha sido commitado no git.
- ☐ Migrar o login para Firebase Authentication (e-mail/senha) real.
- ☐ Parar de gravar `passwordHash` no Firestore; controlar sessão por token.

Alta prioridade (P1)

- ☐ Implementar leitura real da Google Sheets API na Service Account (backend).
- ☐ Unificar a fonte de dados (Firestore *ou* API Express) e remover o caminho duplicado.
- ☐ Substituir todos os números mock por dados reais validados com a Uniodonto.
- ☐ Adicionar Vitest + React Testing Library com testes dos fluxos principais.
- ☐ Configurar CI (GitHub Actions) rodando lint + testes.

Média prioridade (P2)

- ☐ Configurar ESLint + Prettier.
- ☐ Refatorar `Configuracoes.tsx` em componentes menores.
- ☐ Remover `console.log` e código morto (rotas mock não usadas).
- ☐ Adicionar Error Boundary global e trocar `alert()` por toasts.
- ☐ Melhorar acessibilidade (aria-labels, alt, foco por teclado, contraste).
- ☐ Limpar o repositório: remover `firebase-debug.log` e `.tmp-*.png` do git.
- ☐ Revisar estados de carregando/vazio/erro em todas as telas.

Definição de "pronto para entrega": todos os itens P0 concluídos, a integração de dados real funcionando (P1) e ao menos um conjunto mínimo de testes cobrindo login e dashboard. Com isso, a nota geral projetada sobe de **5,1** para a faixa de **8,0–8,5**.